

Python Backend Learning Journey using Flask

1. Project Overview

The **Python Backend Learning Journey using Flask** is a basic Flask web application developed to understand the fundamentals of backend web development using the Flask framework.

The project demonstrates how a Flask application is structured using routes, HTML templates, Jinja2 template inheritance, and static CSS files. The application contains multiple pages that present my Python Backend learning journey, the technologies I have learned, and the concepts I plan to explore next.

This project helped me move from learning Python backend concepts theoretically to applying them by building a simple web application using Flask.

2. Problem Statement

The objective of this project is to build a basic multi-page web application using Flask and understand how a Python backend framework connects routes with HTML templates.

The project focuses on:

- Setting up a Flask development environment
- Creating and managing multiple routes
- Rendering HTML templates using Flask
- Using Jinja2 template inheritance
- Organizing static CSS files
- Connecting multiple webpages through navigation
- Building a responsive and structured web interface

3. Features

Multi-Page Flask Application

- Home page
- About page
- Contact page
- Navigation between multiple pages

Flask Routing

- Separate routes for each webpage
- URL-based page navigation
- Rendering templates using Flask

Template Inheritance

- Common layout using base.html
- Reusable navigation and page structure
- Jinja2 blocks for individual page content

Responsive User Interface

- Responsive navigation bar
- Hero section
- Technology cards
- Learning roadmap
- Responsive CSS Grid layout
- Hover effects
- Modern styling using CSS

Personalized Learning Journey

- Displays technologies already learned
- Shows current Python Backend learning direction

- Presents future learning goals
- Includes links to Email, GitHub, and LinkedIn

These features are consistent with what is actually implemented in the project

4. Technologies Used

Technology Purpose

Python 3	Backend programming
Flask	Web application framework
HTML5	Webpage structure
CSS3	Styling and responsive design
Jinja2	Template inheritance and rendering
VS Code	Development environment

Web Browser Testing the application

The project specifically uses Flask routing, Jinja2 templates, HTML, and CSS as its main technologies

5. Project Structure

Mini_Project_09_Python_Backend_Learning_using_Flask/

```

|
├── README.md
├── Python Backend Learning Journey using Flask.pdf
├── requirements.txt
├── app.py
|
└── app/
    |

```

```
├── __init__.py
├── routes.py
|
├── templates/
|   ├── base.html
|   ├── home.html
|   ├── about.html
|   └── contact.html
|
└── static/
    └── style.css
```

Note: The virtual environment is used locally for development but is not included in the GitHub repository because it is ignored through .gitignore.

6. Python & Flask Concepts Used

Flask

- Flask application initialization
- URL routing
- Route functions
- Template rendering
- Debug mode
- Development server

Jinja2

- Template inheritance
- Base templates
- Blocks

- Reusable webpage structure
- Dynamic navigation

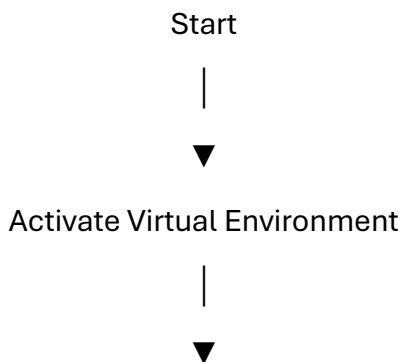
HTML

- Semantic HTML elements
- Navigation
- Sections
- Cards
- Links
- Page structure

CSS

- Flexbox
- CSS Grid
- Responsive layouts
- Hover effects
- Gradients
- Shadows
- Buttons
- Responsive navigation

7. Project Workflow



Install Flask Dependencies



Run app.py



Initialize Flask Application



Load Application Routes



User Visits a URL



Flask Matches the Route



Render HTML Template

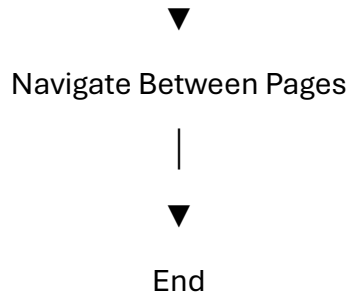


Load Static CSS



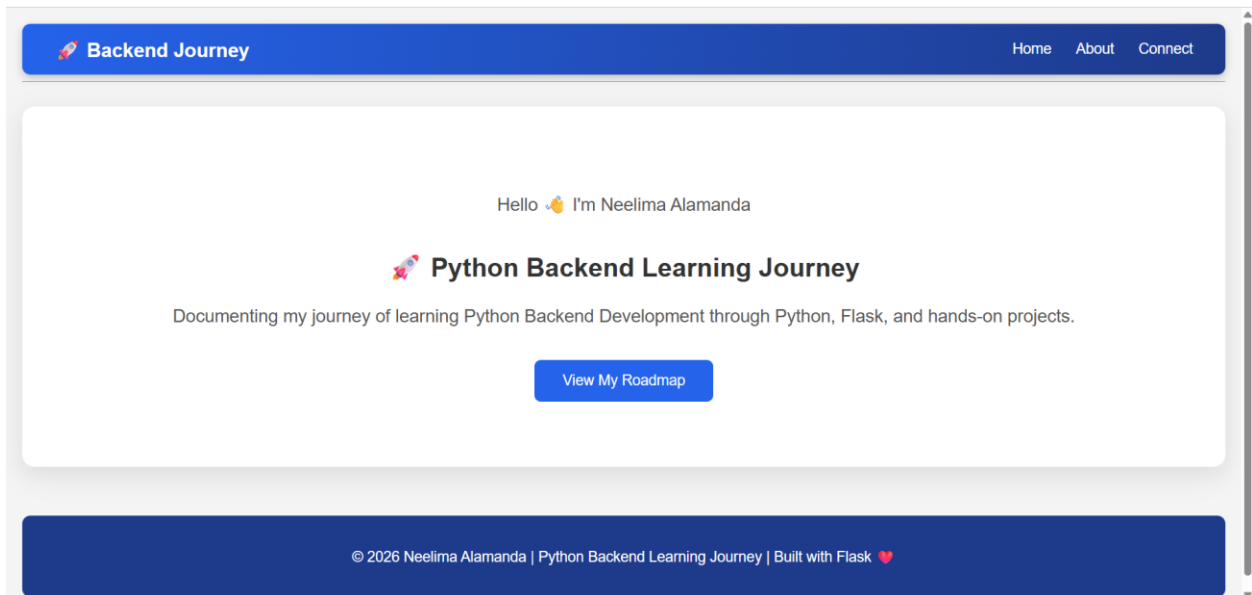
Display Webpage





8. Output Screenshots

Screenshot 1: Home Page

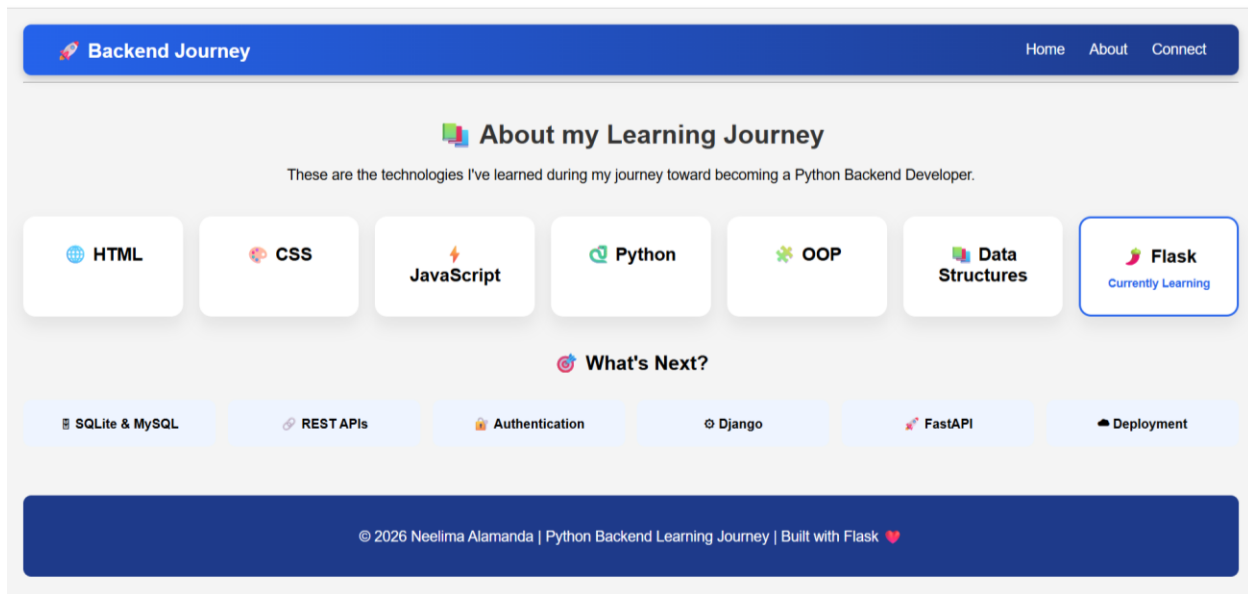


Explanation:

The Home page acts as the landing page of the application. It introduces my Python Backend learning journey through a hero section, a short introduction, and navigation to the other pages.

It demonstrates Flask template rendering along with the responsive design created using HTML and CSS

Screenshot 2: About Page

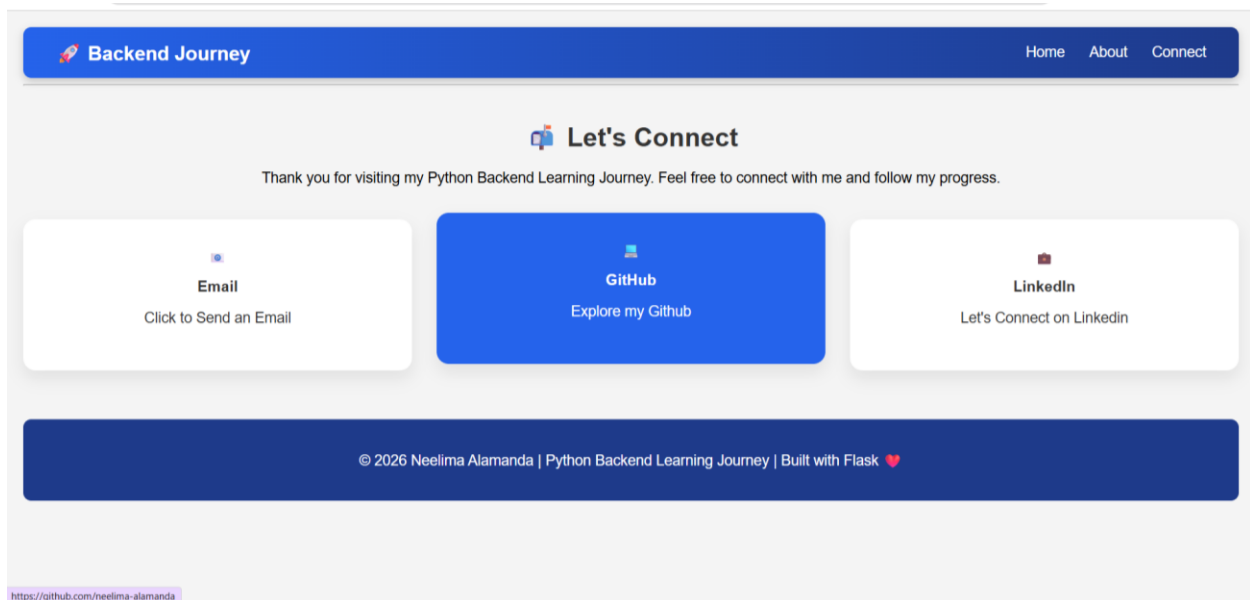


Explanation:

The About page presents the technologies I have learned during my development journey, including HTML, CSS, JavaScript, Python, Object-Oriented Programming, Data Structures, and Flask.

It also presents my future learning roadmap, including areas such as SQLite, REST APIs, Authentication, Django, FastAPI, and deployment.

Screenshot 3: Contact Page

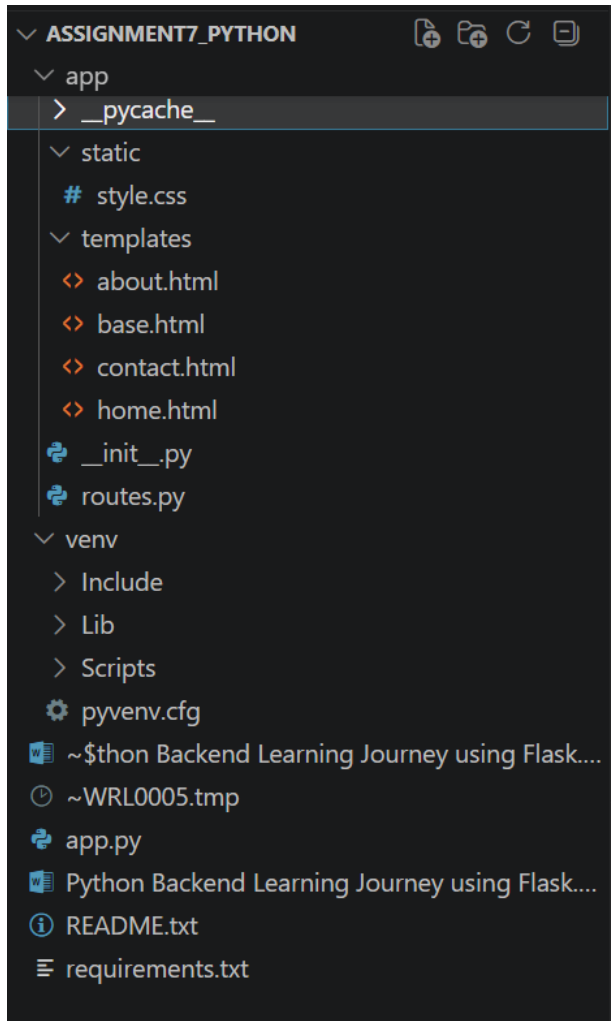


Explanation:

The Contact page provides different ways to connect with me through Email, GitHub, and LinkedIn.

Each contact option is presented through an interactive card with hover effects, making the page simple, responsive, and user-friendly.

Screenshot 4: Project Folder Structure

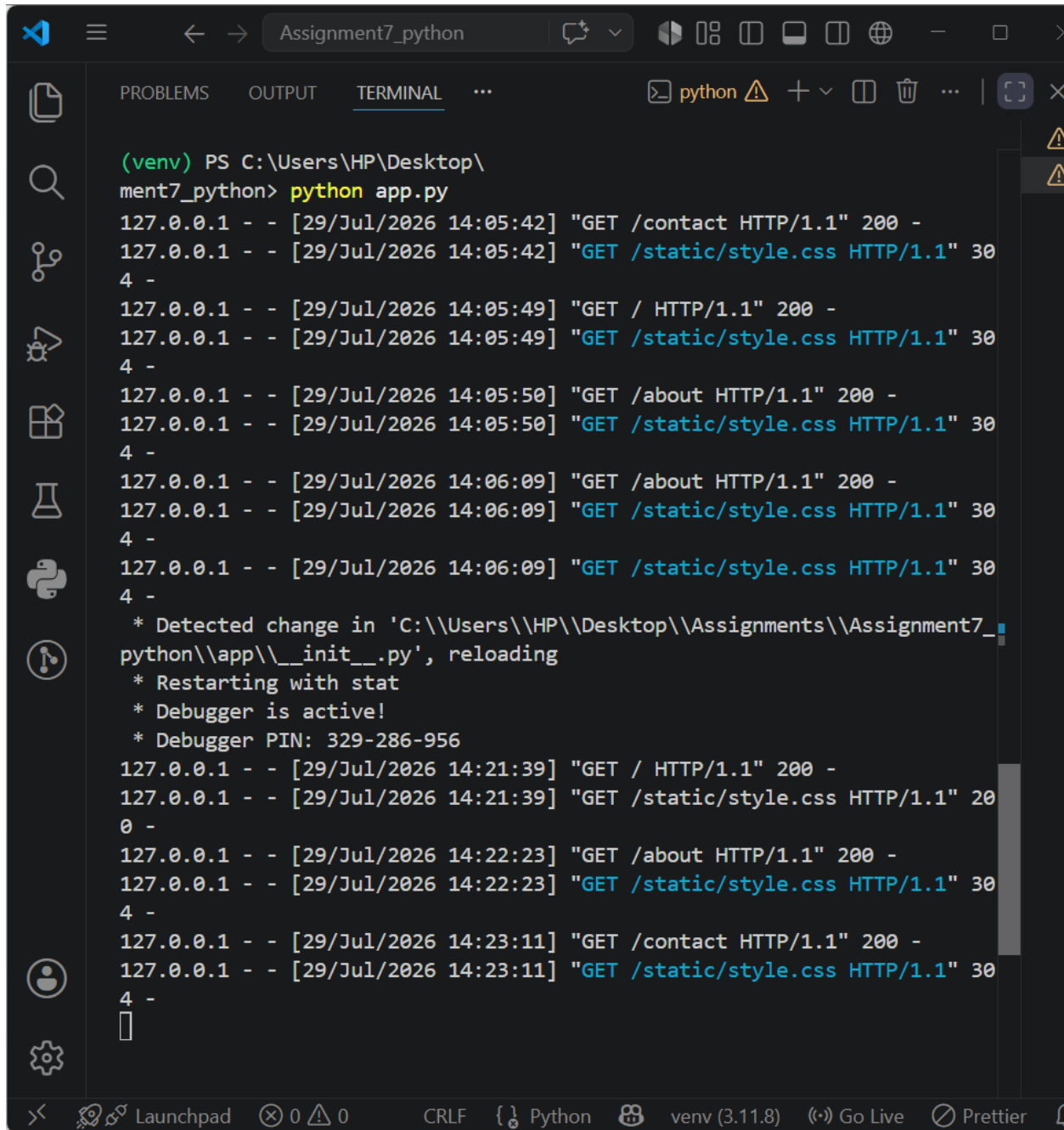


Explanation:

This screenshot shows the organized Flask project structure. The application separates Python application files, HTML templates, and static CSS files into appropriate folders.

This structure makes the application easier to understand, maintain, and extend.

Screenshot 5: Flask Application Running



```
(venv) PS C:\Users\HP\Desktop\
ment7_python> python app.py
127.0.0.1 - - [29/Jul/2026 14:05:42] "GET /contact HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:05:42] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [29/Jul/2026 14:05:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:05:49] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [29/Jul/2026 14:05:50] "GET /about HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:05:50] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [29/Jul/2026 14:06:09] "GET /about HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:06:09] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [29/Jul/2026 14:06:09] "GET /static/style.css HTTP/1.1" 304 -
* Detected change in 'C:\\Users\\HP\\Desktop\\Assignments\\Assignment7_
python\\app\\__init__.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 329-286-956
127.0.0.1 - - [29/Jul/2026 14:21:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:21:39] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:22:23] "GET /about HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:22:23] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [29/Jul/2026 14:23:11] "GET /contact HTTP/1.1" 200 -
127.0.0.1 - - [29/Jul/2026 14:23:11] "GET /static/style.css HTTP/1.1" 304 -
█
```

Explanation:

This screenshot shows the Flask development server running successfully after executing `app.py`.

The terminal confirms that the application is hosted locally at:

`http://127.0.0.1:5000`

This indicates that the Flask application has started successfully and is ready to serve the webpages.

9. Learning Outcomes

After completing this project, I learned how to:

- Understand the basic structure of a Flask application
- Create and manage URL routes
- Render HTML templates using Flask
- Use Jinja2 template inheritance
- Create reusable webpage layouts
- Organize templates and static files
- Connect multiple webpages through Flask routes
- Apply HTML and CSS to a Flask application
- Build a responsive multi-page web application
- Understand how a Python backend framework connects with the frontend
- Organize a project using a maintainable folder structure
- Gain practical experience with Flask development

10. Conclusion

The **Python Backend Learning Journey using Flask** project helped me understand the fundamentals of Flask and how a Python backend framework can be used to build a web application.

By working with Flask routes, Jinja2 templates, HTML, CSS, and static files, I gained practical experience in creating a structured multi-page web application.

This project also gave me a foundation for moving toward more advanced backend concepts such as databases, REST APIs, authentication, Django, FastAPI, and deployment.